

ARMOR: Adaptive Curriculum Meta-Learning for Noise-Robust RAG Reasoning

Yan Wang^{1,2*}, Yuxin Zhang^{1,2*}, Shenyu Zhang^{1,2}, Yongrui Chen^{1,2}, Sheng Bi³, Guilin Qi^{1,2†}

¹School of Computer Science and Engineering, Southeast University, China

²Laboratory of New Generation Artificial Intelligence Technology and Its Interdisciplinary Applications (Southeast University), Ministry of Education, China

³School of Law, Southeast University, China
{yanwangstu, zzyx_cs, gqi}@seu.edu.cn

Abstract

Retrieval-Augmented Generation (RAG) systems have demonstrated remarkable effectiveness in mitigating hallucinations by incorporating external knowledge. However, the retrieval process inevitably introduces noise, posing significant challenges to RAG robustness. Fundamentally, noise robustness is a composite competency encompassing noise identification, retrieval filtering, and robust reasoning. Unfortunately, existing methods often fail to capture this holistic nature, typically restricted to single-dimensional enhancements or hampered by noise-induced optimization interference. To address these limitations, we decompose RAG robustness into three sub-tasks: knowledge boundary perception, retrieval verification, and robust reasoning. Accordingly, we propose ARMOR, an Addaptive Curriculum Meta-Learning for Noise-Robust RAG Reasoning. Inspired by the teaching process, ARMOR introduces an adaptive task scheduler as a *Teacher* to curate training plans according to the absolute learning progress of RAG generators (*Student*). Distinct from GRPO, ARMOR utilizes inter-group advantage estimation during learning to effectively mitigate mode collapse caused by limited output spaces. Experiments on three complex reasoning datasets (*2Wiki-MultiHopQA*, *MusiQue*, and *HotPotQA*) indicate that ARMOR achieves synergistic improvements in retrieval noise robustness, significantly outperforming baselines in output quality. Our code and datasets are available at the link.

1 Introduction

Retrieval-Augmented Generation (RAG) has emerged as a critical paradigm for mitigating hallucinations in Large Language Models (LLMs) by incorporating external knowledge [Lewis *et al.*, 2020; Gao *et al.*, 2023]. However, the retrieval process is inherently imperfect and inevitably introduces irrelevant or misleading *Retrieval Noise*, which poses

a severe challenge to the robustness of RAG systems [Cunasu *et al.*, 2024; Wu *et al.*, 2025]. Fundamentally, noise robustness is not a standalone trait but a composite capability. It requires generative systems to simultaneously perform across three distinct dimensions: identifying knowledge boundaries, filtering noise from retrieved context, and executing correct reasoning [Thakur *et al.*, 2024].

Unfortunately, current methods often focus on specific facets of noise robustness [Chen *et al.*, 2024]. Some studies employ supervised fine-tuning to train generators to detect noise and execute robust reasoning within noisy contexts [Zhou *et al.*, 2025; Wei *et al.*, 2025]. Conversely, others leverage generative systems’ intrinsic knowledge boundary perception to reduce unnecessary retrieval [Su *et al.*, 2024; Wang *et al.*, 2023]. However, these methods generally restrict optimization to isolated capabilities, lacking the synergy required for comprehensive robustness. Reinforcement Learning (RL) offers a paradigm to address this by enabling the co-evolution of these capabilities via end-to-end policy optimization [Shao *et al.*, 2024; Feng *et al.*, 2025].

Nevertheless, applying standard RL to enhance retrieval noise robustness poses two critical challenges. First, the reward signals derived from the multi-dimensional optimization objectives of noise robustness may result in mutual interference or cancellation, thereby preventing the generative system from converging to an optimal policy for any specific capability [Li *et al.*, 2025]. Second, these capabilities exhibit varying degrees of complexity and follow an underlying sequential logic. Just as *a student must learn addition and subtraction before multiplication and division*, the optimization of noise robustness necessitates a sequential learning trajectory. However, standard RL algorithms typically disregard this inherent progression, attempting to optimize all noise robustness capabilities simultaneously [Huang *et al.*, 2025]. This simultaneous learning strategy often induces an overwhelming optimization burden, restricting the generative system from effectively learning these capabilities.

To address these challenges, we propose ARMOR, an Addaptive Curriculum Meta-Learning for Noise-Robust RAG Reasoning. ARMOR leverages curriculum-based Reinforcement Learning to enhance the comprehensive noise robustness of generative systems. Accordingly, we decouple the composite objective of noise robustness into three sub-tasks:

*Equal contributors.

†Corresponding author.

(1) knowledge boundary perception, (2) retrieval verification, and (3) robust reasoning. Specifically, knowledge boundary perception determines whether the model’s intrinsic knowledge suffices or external retrieval is required. Retrieval verification distinguishes relevant evidence from irrelevant noise within the retrieved context. Robust reasoning integrates valid information to generate correct answers even in noisy contexts. Systematically acquiring these capabilities is essential for establishing a resilient defense against retrieval noise.

We draw inspiration from the human teaching process, where teachers dynamically adjust the curriculum based on the student’s learning feedback [Portelas *et al.*, 2019]. ARMOR models the training process as an adaptive curriculum meta-learning framework, and introduces an adaptive task scheduler as a *teacher*. In each training step, the teacher dynamically selects a specific sub-task for the student (the RAG generator), leveraging Absolute Learning Progress (ALP) to quantify the student’s real-time proficiency and prioritize the capability with the highest potential for improvement.

By prioritizing the most critical capability at each stage, ARMOR allows the optimizer to focus on a single dimension of robustness, effectively avoiding the mutual interference and cancellation of reward signals. To further stabilize this decoupled training, we introduce inter-group advantage estimation. Distinct from standard group relative policy optimization (GRPO) [Shao *et al.*, 2024], inter-group advantage estimation extends advantage estimation to a batch-level global scope. By combining inter-group advantage estimation with the adaptive curriculum meta-learning framework, ARMOR effectively eliminates task heterogeneity between different inputs and mitigates mode collapse stemming from limited output spaces. Experiments on three complex reasoning datasets (2WikiMultiHopQA [Ho *et al.*, 2020], MusiQue [Trivedi *et al.*, 2022], and HotPotQA [Yang *et al.*, 2018]) indicate that ARMOR achieves synergistic improvements in retrieval noise robustness, significantly outperforming baselines in output quality.

We summarize our contributions as follows:

- We propose ARMOR, utilizing an ALP-based adaptive task scheduler to dynamically adjust the learning curriculum based on the student’s feedback.
- We introduce inter-group advantage estimation to stabilize training and mitigate mode collapse stemming from limited output spaces.
- Experiments on three noise-injected complex reasoning datasets demonstrate that ARMOR achieves synergistic improvements in retrieval noise robustness.

2 Related Work

Noise Robustness in RAG. Enhancing the robustness of RAG systems against retrieval noise has become a pivotal research direction. A primary defense against noise involves explicit mechanisms to evaluate and filter retrieved documents before or during generation. CRAG [Yan *et al.*, 2024] employs a lightweight retrieval evaluator to assess document quality, triggering corrective actions when the confidence is low. Similarly, TRUSTRAG [Zhou *et al.*, 2025] utilizes clus-

tering and self-assessment to identify and filter out adversarial or misleading information. Rather than filtering inputs, CHAIN-OF-NOTE [Yu *et al.*, 2024] trains models to generate sequential reading notes, explicitly assessing document pertinence before formulating an answer. INSTRUCTRAG [Wei *et al.*, 2025] further advances this by instructing models to synthesize rationales that explain how the answer is derived from noisy contexts. Additionally, RAAT [Fang *et al.*, 2024] employs adaptive adversarial training, exposing the model to various noise types to internalize robust reasoning capabilities. SKR [Wang *et al.*, 2023] utilizes the model’s self-knowledge to classify questions as “known” or “unknown” before invoking a retriever. Similarly, DRAGIN [Su *et al.*, 2024] introduces Real-time Information Needs Detection to trigger retrieval only when the model exhibits uncertainty during generation. However, existing methods typically address noise robustness as an isolated capability by focusing solely on filtering, reasoning, or retrieval triggering.

RL for Robust RAG. Recent advancements have increasingly leveraged RL to enhance the robustness of RAG systems against retrieval noise. Some studies focus on optimizing the interaction policy between reasoning and retrieval. For instance, SEARCH-R1 [Jin *et al.*, 2025] and R1-SEARCHER++ [Song *et al.*, 2025] utilize outcome-based RL to train models to autonomously search queries within reasoning steps, enabling dynamic knowledge acquisition. To enhance noise filtering, R3-RAG [Li *et al.*, 2025] employs a relevance-based process reward to verify retrieved documents at each step. Similarly, LETS [Zhang *et al.*, 2025a] integrates knowledge redundancy and matching rewards to penalize irrelevant retrieval and prevent error propagation in multi-hop reasoning. EVINOTE-RAG [Dai *et al.*, 2025] restructures the workflow into a “retrieve-note-answer” paradigm, using an evidence quality reward to filter noise and distill key evidence before generation. Furthermore, RAG-RL [Huang *et al.*, 2025] investigates static curriculum strategies to improve model robustness against distractors. In contrast, ARMOR decomposes robustness into three distinct sub-tasks, utilizing an ALP-based adaptive task scheduler to dynamically adjust the learning curriculum based on the generator’s feedback.

3 Preliminaries

Task Definition: We focus on the task of RAG under noisy retrieval conditions. Given a QA pair $(q, a) \in \mathcal{T}_{train}$, the RAG system utilizes a retriever to fetch a set of external documents $\mathcal{D} = \{d_1, d_2, \dots, d_k\}$ based on the question q from a large corpus. The retrieved set $\mathcal{D}$ inevitably contains noise due to the inherent imperfections of the retriever. We denote the subset of golden documents as $\mathcal{D}^+ \subseteq \mathcal{D}$ and the noisy documents as $\mathcal{D}^- = \mathcal{D} \setminus \mathcal{D}^+$. The robust RAG generator π_θ aims to generate the correct answer a by effectively filtering $\mathcal{D}^-$ and reasoning over $\mathcal{D}^+$ in conjunction with its intrinsic knowledge. We formulate this as maximizing the probability of the correct answer a given the query q and retrieved set $\mathcal{D}$:

$$\theta^* = \arg \max_{\theta} \sum_{(q,a) \in \mathcal{T}_{train}} \log P_\theta(a | q, \mathcal{D}). \quad (1)$$

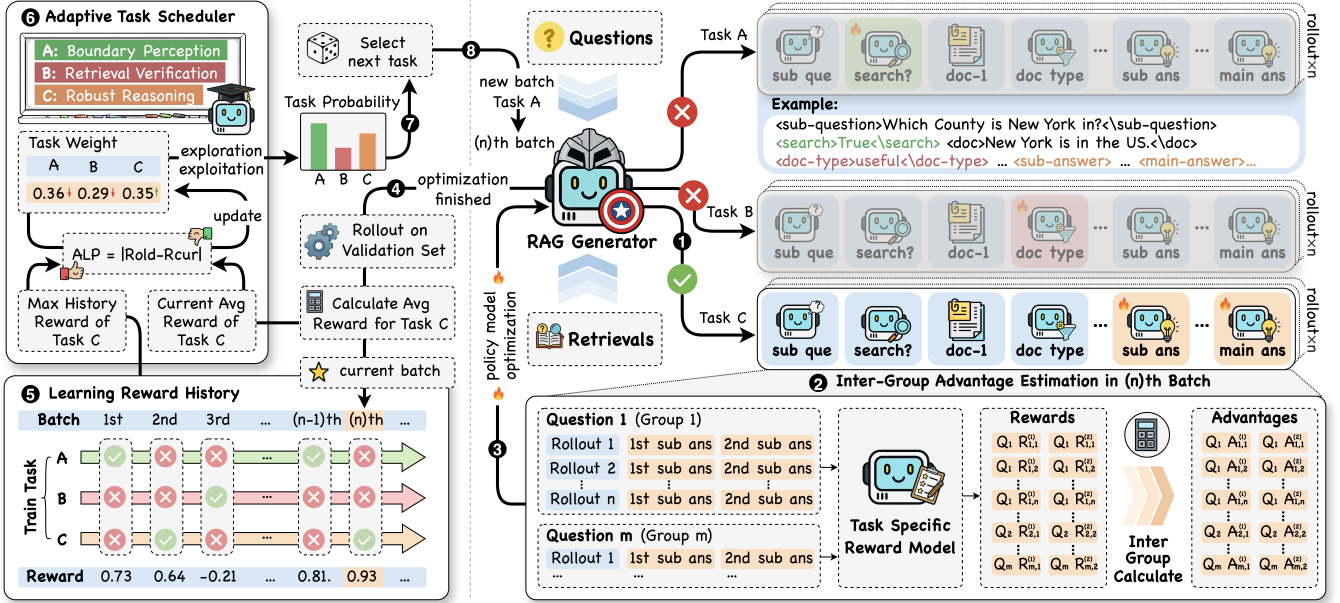


Figure 1: The overall framework of ARMOR. **(Left)** The Adaptive Task Scheduler functions as a *Teacher*, dynamically prioritizing one of the three robustness sub-tasks (Knowledge Boundary Perception, Retrieval Verification, and Robust Reasoning) for each training batch based on the Absolute Learning Progress (ALP). **(Right)** The RAG Generator (*Student*) generates multiple rollouts to learn the selected task. **(Bottom Right)** Inter-Group Advantage Estimation computes reward advantages across the global batch scope rather than within groups, effectively mitigating mode collapse and stabilizing the decoupled training process.

Warm-Up: To equip the RAG generator with the fundamental capabilities of problem decomposition and step-by-step reasoning, we employ SFT on reconstructed multi-hop RAG datasets as a warm-up stage. This phase lays the foundation for the subsequent RL phase. For each pair of `<main-question>` and `<main-answer>`, we construct a reasoning chain to serve as the training target. A single reasoning chain consists of structured nodes: `<think>`, `<sub-question>`, `<search>`, `<doc>`, `<doc-type>`, and `<sub-answer>`. Specifically, we train the generator to decompose the `<main-question>` into sub-questions, determine retrieval policies via `<search>True<search>` or `<search>False<search>`, verify retrieved documents using `<doc-type>` tags, and generate corresponding sub-answers based on the reasoning process. This structured supervision ensures that the generator explicitly internalizes the “retrieve-verify-reason” workflow before exploring complex noisy environments. Detailed dataset construction procedures are provided in Appendix A.

4 Methodology

To achieve comprehensive noise robustness, we propose ARMOR, an Adaptive Curriculum Meta-Learning framework. As illustrated in Figure 1, ARMOR consists of two core components: (1) Adaptive Task Scheduler that dynamically curates the learning curriculum based on the learning feedback, and (2) Inter-Group Advantage Estimation mechanism that stabilizes optimization across heterogeneous sub-tasks.

4.1 Adaptive Curriculum Meta-Learning

We decouple the composite objective of noise robustness into three sub-tasks: Knowledge Boundary Perception (τ_{ret}), Retrieval Verification (τ_{fil}), and Robust Reasoning (τ_{ans}). To avoid the mutual interference of reward signals from distinct tasks, we employ an adaptive task scheduler. In each training round n , the scheduler selects a task τ_s from the task set $\mathcal{T} = \{\tau_{\text{ret}}, \tau_{\text{fil}}, \tau_{\text{ans}}\}$ for optimization, prioritizing tasks with high potential for improvement based on learning feedback. We initialize the task weights $w_1(\tau) = 1/|\mathcal{T}|$ for all $\tau \in \mathcal{T}$. In the n -th round, the selection probability $p_n(\tau)$ is computed to balance exploration and exploitation:

$$\gamma = \min \left\{ 1, \sqrt{\frac{|\mathcal{T}| \ln(|\mathcal{T}|T)}{T}} \right\}, \quad (2)$$

$$p_n(\tau) = (1 - \gamma) \frac{w_n(\tau)}{\sum_{\tau' \in \mathcal{T}} w_n(\tau')} + \frac{\gamma}{|\mathcal{T}|}, \quad (3)$$

where T is the predefined training rounds, and γ serves as an exploration coefficient to prevent premature convergence to local optima. The scheduler samples a task $\tau_s \sim p_n(\cdot)$ at the round n . Upon completing the training for the selected task τ_s , we quantify the model’s increase in competence using Absolute Learning Progress (ALP) [Portelas *et al.*, 2019]. A higher ALP indicates that the task offers significant potential for performance improvement. The raw ALP is defined as:

$$ALP(\tau) = \mathbb{I}[\tau = \tau_s] \cdot |r_n(\tau_s) - r_{\text{old}}(\tau_s)|, \quad (4)$$

where $r_n(\tau_s)$ denotes the task reward received in the current training round n (detailed in section 4.3), $r_{\text{old}}(\tau_s)$ represents the maximum historical reward achieved for task τ_s .

To correct for the bias introduced by probabilistic sampling, we compute the unbiased reward estimation $\hat{r}_n(\tau)$:

$$\hat{r}_n(\tau) = \frac{ALP(\tau)}{p_n(\tau)}, \quad (5)$$

detailed proofs are provided in Appendix B.2. Finally, the scheduler updates the task weights for the next round, incorporating a forgetting factor $\alpha = 1/T$ to smooth updates and retain historical experience:

$$w_{n+1}(\tau) = w_n(\tau) \cdot \exp\left(\frac{\gamma \hat{r}_n(\tau)}{|\mathcal{T}|}\right) + \frac{e\alpha}{|\mathcal{T}|} \cdot \sum_{\tau' \in \mathcal{T}} w_n(\tau'). \quad (6)$$

4.2 Inter-Group Advantage Estimation

Standard Group Relative Policy Optimization (GRPO) normalizes advantages within a group of rollouts corresponding to a single input. However, intra-group normalization results in unstable learning signals for individual tasks during rollouts. In our decoupled training setting, the constrained output space induces high similarity among rollouts generated from the same input, which reduces intra-group diversity and exacerbates the risk of mode collapse. To overcome this limitation, ARMOR introduces Inter-Group Advantage Estimation, which performs advantage normalization across the entire training batch rather than within isolated inputs.

Given a batch of questions $\mathcal{Q} = \{q_1, \dots, q_m\}$, we generate l rollouts for each question, denoted as:

$$\mathcal{O}_{\text{train}} = \{o_{i,j} \mid 1 \leq i \leq m, 1 \leq j \leq l\}, \quad (7)$$

where $o_{i,j}$ denotes the j -th rollout of question q_i . For the selected task τ_s , each rollout $o_{i,j}$ receives a sequence of task-level rewards, denoted as:

$$\mathbf{r}_{i,j} = [r_{i,j}^{(1)}, \dots, r_{i,j}^{(K_{i,j})}]^\top, \quad (8)$$

where $r_{i,j}^{(k)}$ represents the reward of k -th task span in the rollout $o_{i,j}$. In addition, each rollout sequence $o_{i,j}$ is associated with a format reward $r_{i,j}^{\text{fmt}}$, which evaluates whether the rollout sequence conforms to the required format. We define the global sets of task and format rewards in current batch as:

$$R_{\text{task}} = \{\mathbf{r}_{i,j} \mid 1 \leq i \leq m, 1 \leq j \leq l\}, \quad (9)$$

$$R_{\text{fmt}} = \{r_{i,j}^{\text{fmt}} \mid 1 \leq i \leq m, 1 \leq j \leq l\}. \quad (10)$$

As shown in Figure 1, we normalize rewards using the global statistics of the entire batch to compute the task-level advantage $A_{i,j}^{\text{task}(k)}$ and format-level advantage $A_{i,j}^{\text{fmt}}$:

$$A_{i,j}^{\text{task}(k)} = \frac{r_{i,j}^{(k)} - \text{mean}(R_{\text{task}})}{\text{std}(R_{\text{task}})}, \quad (11)$$

$$A_{i,j}^{\text{fmt}} = \frac{r_{i,j}^{\text{fmt}} - \text{mean}(R_{\text{fmt}})}{\text{std}(R_{\text{fmt}})}, \quad (12)$$

where $\text{mean}(\cdot)$ and $\text{std}(\cdot)$ compute the mean and standard deviation over the entire batch. By combining inter-group advantage estimation with the adaptive curriculum meta-learning framework, ARMOR effectively eliminates task heterogeneity between different inputs and provides a more stable training signal.

The final token-level inter-group advantage $A_{i,j}^{(t)}$ combines dual-level signals:

$$A_{i,j}^{(t)} = \sum_{k=1}^{K_{i,j}} \mathbb{I}[t \in \text{tokens}(i, j, k)] \cdot A_{i,j}^{\text{task}(k)} + \omega A_{i,j}^{\text{fmt}} \quad (13)$$

where t is the token index in rollout $o_{i,j}$, $\text{tokens}(i, j, k)$ denotes the token index of the k -th task span in rollout $o_{i,j}$. $A_{i,j}^{(t)}$ represents the token advantage assigned to token t in rollout $o_{i,j}$. The weighting coefficient ω controls the relative strength of the format signal. Then the clipped policy optimization objective of ARMOR is as follows:

$$\begin{aligned} \mathcal{J}_{\text{ARMOR}}(\theta) = & \mathbb{E} \left[q_i \in \mathcal{Q}, \{o_{i,j}\}_{j=1}^l \sim \pi_{\theta_{old}}(\cdot \mid q_i) \right] \frac{1}{\sum_{j=1}^l |o_{i,j}|} \sum_{j=1}^l \sum_{t=1}^{|o_{i,j}|} \\ & \min \left[\frac{\pi_{\theta}(o_{i,j}^{(t)} \mid q, o_{i,j}^{(<t)})}{\pi_{\theta_{old}}(o_{i,j}^{(t)} \mid q, o_{i,j}^{(<t)})} A_{i,j}^{(t)}, \right. \\ & \left. \text{clip} \left(\frac{\pi_{\theta}(o_{i,j}^{(t)} \mid q, o_{i,j}^{(<t)})}{\pi_{\theta_{old}}(o_{i,j}^{(t)} \mid q, o_{i,j}^{(<t)})}, 1 - \varepsilon_{low}, 1 + \varepsilon_{high} \right) A_{i,j}^{(t)} \right]. \end{aligned} \quad (14)$$

After the training for the task τ_s in the n -th batch, the task reward $r_n(\tau_s) = \text{mean}(R_{\text{task}})$ is calculated on the validation dataset, and returned to the adaptive task scheduler to update the task weights.

4.3 Reward Modeling

To precisely guide the generator toward robust behaviors, we design tailored reward functions for each sub-task, addressing their specific optimization challenges.

Knowledge Boundary Perception. The primary objective of this task is to enable the generator to accurately perceive the boundaries of its intrinsic knowledge. For each generated sub-question `<sub-question>`, the RAG generator predicts a `<search>` tag to determine whether to invoke the retriever. The action space is defined as:

$$\mathcal{C}_{\text{retriever}} = \{\text{true}, \text{false}\} \quad (15)$$

To align the model’s retrieval behavior with the actual necessity of external information, we design a reward function that evaluates whether the model should retrieve from external knowledge based on the correctness of its direct answer. We denote the correctness of the answer generated without retrieval as $v_{\text{ans}} \in \{\text{correct}, \text{error}\}$, and the model’s chosen retrieve action as a_{ret} . The reward r_{ret} is defined as follows:

$$r_{\text{ret}} = \begin{cases} 1 & \text{if } v_{\text{ans}} \text{ is error} \wedge a_{\text{ret}} = \text{true}, \\ -1 & \text{if } v_{\text{ans}} \text{ is error} \wedge a_{\text{ret}} = \text{false}, \\ -0.5 & \text{if } v_{\text{ans}} \text{ is correct} \wedge a_{\text{ret}} = \text{true}, \\ 0.5 & \text{if } v_{\text{ans}} \text{ is correct} \wedge a_{\text{ret}} = \text{false}. \end{cases} \quad (16)$$

Retrieval Verification. This task requires the generator to classify each retrieved document into three categories: $\mathcal{C}_{\text{doc}} = \{\text{golden}, \text{inconsequential}, \text{irrelevant}\}$ [Zhang *et al.*, 2025b]. Standard coarse-grained binary rewards (correct/incorrect classification) often lead to training instability. To mitigate this, we propose a fine-grained Retrieval Verification Reward based on the probability margin. For a document d , let c^* be the ground-truth label and $p(c)$ be the predicted

probability of class c . We first calculate the confidence margin $m(d)$ between the target class and the most competitive incorrect class:

$$m(d) = p(c^*) - \max_{c \neq c^*, c \in \mathcal{C}_{doc}} p(c). \quad (17)$$

The final retrieval verification task reward is:

$$r_{\text{fil}}(d) = \tanh(\beta \cdot m(d)), \quad (18)$$

where scaling factor β is used to control the sensitivity of the reward scaling.

Robust Reasoning. The goal of robust reasoning is to generate correct sub-answers `<sub-answer>` and a final answer `<final-answer>` given noisy context. We compute the answer reward r_{ans} by first obtaining a raw correctness score s_{ans} from an LLM evaluator, then linearly mapping it to the range $[-1, 1]$:

$$r_{\text{ans}} = 2 \cdot \frac{s_{\text{ans}} - s_{\text{min}}}{s_{\text{max}} - s_{\text{min}}} - 1, \quad (19)$$

where $[s_{\text{min}}, s_{\text{max}}]$ is the range of correctness score. Details of the LLM-based evaluation are provided in Appendix E.

5 Experiment

In this section, we present empirical evaluations of the effectiveness of ARMOR in defending against retrieval noise across three complex reasoning datasets. Specifically, we aim to validate the following: (1) ARMOR’s capability in enhancing the noise robustness of RAG generators; (2) the contribution of core components via ablation studies; (3) the dynamic trends of learning feedback $p_n(\tau)$ during training.

5.1 Experiment Setup

Datasets. As detailed in Appendix A, we reconstruct *2WikiMultiHopQA* and *MuSiQue* datasets. For each dataset, training set consists of two stages: a warm-up stage and a RL stage. The validation set derives from the reconstructed datasets, while the test set consists of two reconstructed subsets and *HotpotQA*, which serves as an out-of-distribution dataset for evaluating model generalization. The detailed dataset statistics are presented in Table 1.

Subset	2Wiki	MuSiQue	HotpotQA
Train_{Warm-Up}	4,467	496	–
Train_{RL}	7,716	857	–
Test	1,352	375	500
Validation	1,352	375	–

Table 1: Experiment dataset statistics.

Evaluation Metrics. We evaluated the model using the following metrics: (1) **Exact Match (EM)** measures the strict accuracy by determining if the predicted answer matches the ground truth verbatim. (2) **F1-Score (F1)** captures the token-level overlap between prediction and reference, offering flexibility for answers with minor phrasing variations. However, these traditional metrics rely on surface-level matching and

often fail to capture semantic validity in generative tasks. To address this, we introduce (3) **Correctness (Cor.)**, an LLM-based metric utilizing GPT-5 to assess the factual accuracy and completeness of answers against references. For clarity, correctness scores are normalized to the range $[0, 1]$. Detailed evaluation prompts and results aligned with manual annotations are presented in the Appendix E.

Baselines. We compare ARMOR against several established baselines: (1) **Adaptive Retrieval:** We compare against SKR [Wang *et al.*, 2023] and DRAGIN [Su *et al.*, 2024], which selectively trigger retrieval based on model’s intrinsic knowledge boundary to mitigate noise introduction. (2) **Robust Reasoning:** This category includes CHAIN-OF-NOTE [Yu *et al.*, 2024] and IRCOT [Trivedi *et al.*, 2023]. We employ standard CoT without retrieval as a lower bound to benchmark pure reasoning robustness. (3) **RL-Enhanced RAG:** This includes methods like RAG-RL [Huang *et al.*, 2025] and SEARCH-R1 [Jin *et al.*, 2025] that optimize retrieval and reasoning policies via reinforcement learning. In addition, we include NO-RAG (direct inference) and VANILLA-RAG (standard RAG workflow), which serve as performance lower bounds, as they lack specific mechanisms to defend against retrieval noise. We employ two mainstream LLM series with varying parameter scales as the RAG generators: Llama-3.2-1B, Llama-3.2-3B [Team, 2024], Qwen-3-0.6B, Qwen-3-1.7B and Qwen-3-4B [Yang *et al.*, 2025]. For all baselines, we strictly follow the hyperparameter settings and configurations reported in their original studies. Detailed baseline configurations are provided in Appendix C.

Training Details. We train the RAG generator for one epoch using Adam optimizer [Kingma and Ba, 2015] with a learning rate of $5e-5$. One epoch includes 857 training rounds, and each round consists of 10 questions. The clipping coefficient ε_{low} and $\varepsilon_{\text{high}}$ are set to 0.2 and 0.28, separately. The default rollout number l is set to 2. The weighting coefficient ω is set to 1, and the scaling factor β in the retrieval verification task is set to 6. During the rollout process, we employ `e5-base-v2` [Wang *et al.*, 2022] as the retriever, retrieving a maximum of three documents per retrieval triggering. For generation, we adopt a sampling temperature of 0.85, along with $\text{top-}p = 0.95$ and $\text{top-}k = 50$. All reported results are averaged over three independent runs to ensure statistical reliability.

5.2 Main Results

As demonstrated in our preliminary study (Appendix C.2), standard retrieval systems inevitably retrieve irrelevant or inconsequential documents, introducing *natural noise* into the generation process. Table 2 presents the comparative performance of ARMOR and baselines under this realistic setting. ARMOR achieves SOTA performance across all datasets and metrics, demonstrating remarkable consistency on both Qwen and Llama backbones. Specifically, ARMOR surpasses the strongest baseline (IRCOT) by a substantial margin, achieving an average Correctness improvement of 38.8% on Qwen-3-1.7B and 19.9% on Llama-3.2-3B. Standard VANILLA-RAG shows limited improvement over direct inference (NO-RAG) and performs poorly on complex datasets. This confirms that without explicit robustness mechanisms, generators

Method	2WikiMultiHopQA			MusiQue			HotpotQA			Avg.		
	Cor.	F1	EM	Cor.	F1	EM	Cor.	F1	EM	Cor.	F1	EM
Qwen-3-1.7B												
No-RAG	37.72	37.21	32.99	3.73	2.96	0.00	24.20	22.44	15.67	21.88	20.87	16.22
VANILLA-RAG	47.63	44.67	38.46	24.00	27.43	17.33	45.07	45.72	38.33	38.90	39.27	31.37
CoT	39.64	17.13	11.39	8.27	6.27	1.33	22.47	18.03	13.60	23.46	13.81	8.77
CoN	63.61	13.13	4.73	29.33	12.36	5.33	54.53	22.65	17.07	49.16	16.05	9.04
IRCoT	50.59	48.80	41.86	32.00	29.97	18.67	55.40	51.97	46.26	46.00	43.58	35.60
DRAGIN	39.70	39.46	36.98	6.67	8.87	5.33	26.87	26.52	22.73	24.41	24.95	21.68
SKR	35.50	32.00	26.92	5.33	3.86	0.00	21.13	20.78	13.20	20.65	18.88	13.37
RAG-RL	49.70	46.67	41.39	29.66	28.79	20.33	48.33	47.64	42.67	42.56	41.03	34.80
SEARCH-R1	41.57	39.61	37.28	10.67	10.55	3.73	28.27	28.21	21.93	26.84	26.12	20.98
ARMOR (warm-up stage)	70.41	70.69	68.34	38.67	39.04	29.33	47.53	45.20	38.86	52.20	51.64	45.51
ARMOR	79.14	79.22	77.22	54.67	57.22	46.67	57.73	54.86	58.00	63.85	63.77	60.63
Llama-3.2-3B												
No-RAG	28.25	30.09	25.15	5.33	2.32	0.00	22.60	17.98	13.40	18.73	16.80	12.85
VANILLA-RAG	59.32	57.58	53.40	16.00	17.48	10.67	50.27	44.52	36.27	41.86	39.86	33.45
CoT	36.83	17.22	14.20	8.00	3.82	1.33	35.60	23.72	18.07	26.81	14.92	11.20
CoN	51.33	7.80	1.92	14.67	3.71	0.00	47.47	10.63	6.20	37.82	7.38	2.71
IRCoT	61.39	58.76	53.40	34.67	32.43	22.67	60.13	57.77	48.00	52.06	49.65	41.36
DRAGIN	42.60	41.76	39.50	9.33	7.77	4.00	37.47	35.85	29.73	29.80	28.46	24.41
SKR	35.21	36.98	32.99	8.26	6.95	2.67	35.00	32.50	31.33	26.16	25.48	22.33
RAG-RL	64.48	62.58	58.40	29.67	30.21	21.33	56.60	50.47	41.27	50.25	47.75	40.33
SEARCH-R1	42.31	40.76	38.76	6.67	8.50	4.00	26.33	27.22	24.53	25.10	25.49	22.43
ARMOR (warm-up stage)	71.01	71.28	68.93	32.00	36.54	21.33	45.67	47.79	37.60	49.56	51.87	42.62
ARMOR	77.81	78.09	75.59	42.67	45.62	29.33	66.93	66.47	58.67	62.47	63.39	54.53

Table 2: Performance comparison of ARMOR with baselines. We report Correctness (Cor.), F1 score, and Exact Match (EM) in percentage (%) using Qwen-3-1.7B and Llama-3.2-3B as backbones. The best results are highlighted in **bold**. Full results are provided in Appendix D.

are easily misled by the inherent noise in natural retrieval. IRCOT remains competitive by interleaving reasoning and retrieval steps. CoN struggles to maintain stability, suggesting that their sequential noise-filtering logic is prone to error propagation in open-domain settings. ARMOR’s decoupled training objective effectively mitigates this issue. Existing RL-based approaches (RAG-RL, SEARCH-R1) exhibit sub-optimal performance, highlighting the difficulty of optimizing RAG generators with sparse rewards under natural noise.

We conducted a controlled experiment by manually injecting noise into the retrieval. We fixed the number of retrieved documents to 5 and varied the noise ratio from 0% to 100%, as shown in Figure 2. Baseline methods exhibit a **precipitous performance decline** as the noise ratio increases. IRCOT and SEARCH-R1 suffer significant losses, indicating they struggle to filter out distracting information effectively when valid evidence is scarce. In contrast, ARMOR demonstrates exceptional stability, maintaining a high correctness score of 69% even at an 80% noise ratio. ARMOR often outperforms the DIRECT ANSWER baseline or remains comparable at 100% noise. This indicates that ARMOR has effectively learned a *rejection mechanism* via **Inter-Group Advantage Estimation**, allowing it to suppress the influence of irrelevant contexts rather than being hallucinated.

5.3 Ablation Study

We compare ARMOR with the following variants: (1) *w/o CRL*: replaces the ALP-based scheduler with random sam-

pling (*-random*), rigid sequential ordering (*-seq*), or outcome-supervised training (*-outcome sup*: removing process rewards); (2) *w/o Inter-Group Adv*: reverts global advantage normalization to standard intra-group normalization; (3) *Base Setting*: removes both adaptive curriculum and inter-group estimation, reverting to standard outcome-supervised RL. As shown in Table 3, ARMOR consistently outperforms all variant settings across all metrics. Specifically, *w/o CRL -seq* suffers a 47.51% drop in average correctness, indicating that rigid task ordering leads to **catastrophic forgetting** among decoupled sub-tasks. Similarly, *w/o CRL -random* underperforms relative to ARMOR, confirming that the ALP-based scheduler is essential for dynamically balancing exploration and exploitation based on learning feedback. Reverting to standard intra-group normalization (*w/o inter-group adv.*) degrades performance by 14.58%. This decline highlights that standard intra-group normalization **yields unstable signals in our decoupled setting**. In contrast, ARMOR’s batch-level advantage estimation provides robust supervision, effectively preventing mode collapse and stabilizing optimization.

5.4 Analysis of Dynamic Learning Feedback

We visualize the dynamic trends of the task selection probability $p_n(\tau)$ throughout the training process. Figure 3 illustrates these trends for both Llama-3.2-3B and Qwen-3-1.7B backbones. Observing the training dynamics of Qwen-3-1.7B, the scheduler assigns high probability to the Knowledge Boundary Perception task in the early stages. This in-

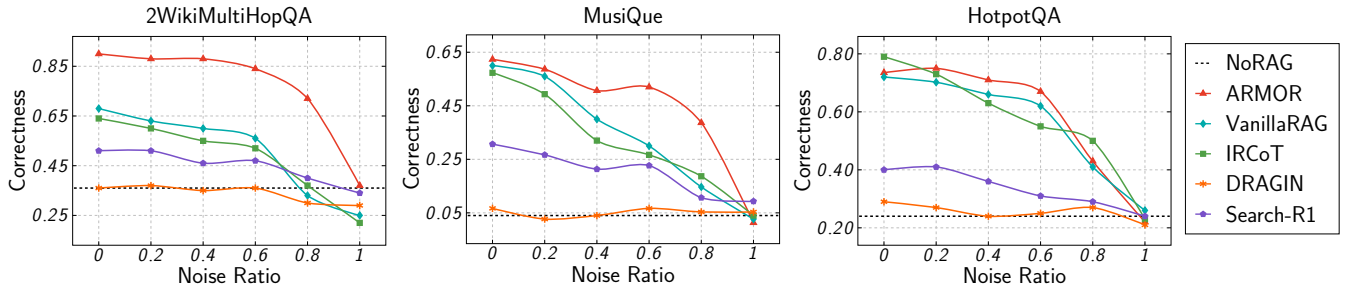


Figure 2: Robustness analysis under controlled noise ratios (from 0% to 100%) with fixed retrieval size of $K = 5$.

Method	2WikiMultiHopQA			MusiQue			HotpotQA			Avg.		
	Cor.	F1	EM	Cor.	F1	EM	Cor.	F1	EM	Cor.	F1	EM
Qwen-3-1.7B												
ARMOR	79.14	79.22	77.22	54.67	57.22	46.67	57.73	54.86	58.00	63.85	63.77	60.63
<i>w/o CRL -random</i>	67.66	67.97	66.48	39.33	40.59	28.67	37.27	35.45	27.33	48.09	48.00	40.83
<i>w/o CRL -seq</i>	52.22	52.69	50.89	23.65	26.50	15.20	24.67	21.76	11.27	33.51	33.65	25.79
<i>w/o CRL -outcome sup.</i>	60.22	60.45	58.59	32.33	32.37	23.00	30.80	27.72	18.73	41.12	40.18	33.44
<i>w/o inter-group adv.</i>	73.55	73.49	71.63	48.00	46.28	40.00	42.07	39.73	31.13	54.54	53.17	47.59
Base Setting	71.63	72.04	70.30	44.33	45.79	36.67	40.47	38.29	30.07	52.14	52.04	45.68

Table 3: Ablation analysis on the effectiveness of the Adaptive Curriculum Learning (CRL) strategy and Inter-Group Advantage Estimation (inter-group adv). The best results are highlighted in **bold**.

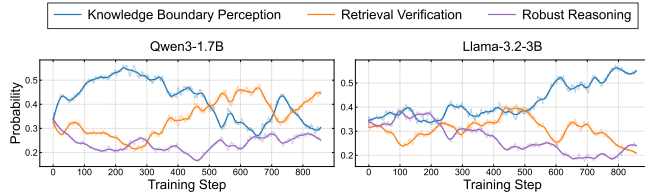


Figure 3: Task selection probabilities $p_n(\tau)$ during the training.

icates that the generator primarily focuses on distinguishing “what it knows” vs. “what it needs to retrieve.” The scheduler shifts its focus toward Retrieval Verification around step 300. This transition suggests that once the generator learns “when to retrieve,” it begins to focus on “how to filter noise” from the retrieval. Meanwhile, the attention on Robust Reasoning remains relatively stable, serving as a consistent regularization throughout the process. The trends for Llama-3.2-3B exhibit a different pattern, highlighting the scheduler’s adaptability to different generator architectures. Unlike Qwen, Llama shows a continuous and increasing demand for Knowledge Boundary Perception. This suggests that for the Llama backbone, accurately determining the necessity of retrieval is a more challenging task. These findings confirm that ARMOR does not impose a rigid “one-size-fits-all” curriculum. Instead, it acts as a dynamic meta-learner, tailoring the training strategy to the learning trajectory of the underlying generator.

5.5 Case Study

We analyze a multi-hop query: “When was Ole Rynning’s father born?” (answer: “May 30, 1778”), detailed case is provided in Appendix D.2. ARMOR successfully decomposes the logic: identifying the father (“Jens Rynning”) first, then

retrieving his birth year. Crucially, ARMOR effectively filters out distractor documents describing the son’s birth “1809”, leading to the correct answer. In contrast, SEARCH-R1 exhibits chaotic reasoning, generating hallucinated contexts and failing to align the answer type. IRCOT suffers from distraction failure, although it retrieves the correct evidence, it lacks explicit noise rejection and is misled by the high semantic overlap of the son’s birth date. This case validates that ARMOR’s adaptive curriculum meta-learning effectively equips the generator with the ability to distinguish golden evidence from noise in complex reasoning scenarios.

6 Conclusion

In this work, we presented ARMOR, a novel framework designed to fortify RAG systems against the pervasive challenge of retrieval noise. ARMOR decouples the training objective into three distinct sub-tasks: Knowledge Boundary Perception, Retrieval Verification, and Robust Reasoning. To coordinate these tasks, ARMOR introduces an Adaptive Curriculum Meta-Learning strategy, where a *teacher* scheduler dynamically prioritizes learning objectives based on the generator’s Absolute Learning Progress. Moreover, ARMOR addresses the optimization instability inherent in decoupled reinforcement learning by introducing Inter-Group Advantage Estimation, which mitigates mode collapse and ensures stable gradient optimization via global batch normalization. Extensive experiments across three complex multi-hop reasoning datasets demonstrate that ARMOR exhibits exceptional resilience, maintaining high correctness even under extreme noise ratios. This confirms that Adaptive Curriculum Scheduling and Inter-Group Advantage Estimation are essential for comprehensive RAG noise-robust reasoning.

7 Limitations

Evaluation Scope. Our experiments is confined to general-domain multi-hop QA datasets, including *2WikiMultiHopQA*, *MuSiQue* and *HotpotQA*. These datasets fail to capture the constraints and structural complexity of specialized domains. Consequently, the effectiveness of ARMOR in practical, high-stakes environments may be overestimated. Extending the evaluation to domains such as mathematics and healthcare is necessary to more rigorously assess its robustness and generalization. **Language Scope.** Our experiments are limited to English-only corpora, which biases the evaluation toward high-resource language settings. Although the proposed approach is not inherently language-dependent, its performance under multilingual conditions—where retrieval quality, linguistic variation, and knowledge distribution differ significantly—remains underexplored. Systematically extending and validating ARMOR in cross-lingual settings is an important direction for future investigation.

Acknowledgments

This work is partially supported by National Nature Science Foundation of China under No. 62476058. We thank the Big Data Computing Center of Southeast University for providing the facility support on the numerical calculations in this paper.

References

- [Chen *et al.*, 2024] Jiawei Chen, Hongyu Lin, Xianpei Han, and Le Sun. Benchmarking large language models in retrieval-augmented generation. In Michael J. Wooldridge, Jennifer G. Dy, and Sriraam Natarajan, editors, *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2014, February 20-27, 2024, Vancouver, Canada*, pages 17754–17762. AAAI Press, 2024.
- [Comanici *et al.*, 2025] Gheorghe Comanici, Eric Bieber, Mike Schaekermann, et al. Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities, 2025.
- [Cuconasu *et al.*, 2024] Florin Cuconasu, Giovanni Trapolini, Federico Siciliano, Simone Filice, Cesare Campagnano, Yoelle Maarek, Nicola Tonellotto, and Fabrizio Silvestri. The power of noise: Redefining retrieval for RAG systems. In Grace Hui Yang, Hongning Wang, Sam Han, Claudia Hauff, Guido Zuccon, and Yi Zhang, editors, *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2024, Washington DC, USA, July 14-18, 2024*, pages 719–729. ACM, 2024.
- [Dai *et al.*, 2025] Yuqin Dai, Guoqing Wang, Yuan Wang, et al. Evinote-rag: Enhancing RAG models via answer-supportive evidence notes. *CoRR*, abs/2509.00877, 2025.
- [DeepSeek-AI, 2025] DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *CoRR*, abs/2501.12948, 2025.
- [Fang *et al.*, 2024] Feiteng Fang, Yuelin Bai, Shiwen Ni, et al. Enhancing noise robustness of retrieval-augmented language models with adaptive adversarial training. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2024, Bangkok, Thailand, August 11-16, 2024*, pages 10028–10039. Association for Computational Linguistics, 2024.
- [Feng *et al.*, 2025] Lang Feng, Zhenghai Xue, Tingcong Liu, and Bo An. Group-in-group policy optimization for LLM agent training. *CoRR*, abs/2505.10978, 2025.
- [Gao *et al.*, 2023] Yunfan Gao, Yun Xiong, Xinyu Gao, et al. Retrieval-augmented generation for large language models: A survey. *CoRR*, abs/2312.10997, 2023.
- [Ho *et al.*, 2020] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing A multi-hop QA dataset for comprehensive evaluation of reasoning steps. In Donia Scott, Núria Bel, and Chengqing Zong, editors, *Proceedings of the 28th International Conference on Computational Linguistics, COLING 2020, Barcelona, Spain (Online), December 8-13, 2020*, pages 6609–6625. International Committee on Computational Linguistics, 2020.
- [Huang *et al.*, 2025] Jerry Huang, Siddarth Madala, Risham Sidhu, et al. RAG-RL: advancing retrieval-augmented generation via RL and curriculum learning. *CoRR*, abs/2503.12759, 2025.
- [Jin *et al.*, 2025] Bowen Jin, Hansi Zeng, Zhenrui Yue, et al. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *CoRR*, abs/2503.09516, 2025.
- [Kingma and Ba, 2015] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In Yoshua Bengio and Yann LeCun, editors, *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015.
- [Lewis *et al.*, 2020] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In Hugo Larochelle, Marc’Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin, editors, *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020.
- [Li *et al.*, 2025] Yuan Li, Qi Luo, Xiaonan Li, Bufan Li, et al. R3-RAG: learning step-by-step reasoning and retrieval for llms via reinforcement learning. *CoRR*, abs/2505.23794, 2025.
- [Portelas *et al.*, 2019] Rémy Portelas, Cédric Colas, Katja Hofmann, and Pierre-Yves Oudeyer. Teacher algorithms for curriculum learning of deep RL in continuously parameterized environments. In Leslie Pack Kaelbling, Danica Kragic, and Komei Sugiura, editors, *3rd Annual Conference on Robot Learning, CoRL 2019, Osaka, Japan, Oc-*

- tober 30 - November 1, 2019, *Proceedings*, volume 100 of *Proceedings of Machine Learning Research*, pages 835–853. PMLR, 2019.
- [Shao *et al.*, 2024] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *CoRR*, abs/2402.03300, 2024.
- [Singh *et al.*, 2025] Aaditya Singh, Adam Fry, Adam Perelman, et al. Openai gpt-5 system card, 2025.
- [Song *et al.*, 2025] Huatong Song, Jinhao Jiang, Wenqing Tian, et al. R1-searcher++: Incentivizing the dynamic knowledge acquisition of llms via reinforcement learning. *CoRR*, abs/2505.17005, 2025.
- [Su *et al.*, 2024] Weihang Su, Yichen Tang, Qingyao Ai, et al. DRAGIN: dynamic retrieval augmented generation based on the real-time information needs of large language models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2024, Bangkok, Thailand, August 11-16, 2024, pages 12991–13013. Association for Computational Linguistics, 2024.
- [Team, 2024] Llama Team. The llama 3 herd of models. *CoRR*, abs/2407.21783, 2024.
- [Thakur *et al.*, 2024] Nandan Thakur, Luiz Bonifacio, Xinyu Zhang, et al. "knowing when you don't know": A multilingual relevance assessment dataset for robust retrieval-augmented generation. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Findings of the Association for Computational Linguistics: EMNLP 2024, Miami, Florida, USA, November 12-16, 2024*, pages 12508–12526. Association for Computational Linguistics, 2024.
- [Trivedi *et al.*, 2022] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop questions via single-hop question composition. *Trans. Assoc. Comput. Linguistics*, 10:539–554, 2022.
- [Trivedi *et al.*, 2023] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In Anna Rogers, Jordan L. Boyd-Graber, and Naoaki Okazaki, editors, *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2023, Toronto, Canada, July 9-14, 2023, pages 10014–10037. Association for Computational Linguistics, 2023.
- [Wang *et al.*, 2022] Liang Wang, Nan Yang, Xiaolong Huang, et al. Text embeddings by weakly-supervised contrastive pre-training. *CoRR*, abs/2212.03533, 2022.
- [Wang *et al.*, 2023] Yile Wang, Peng Li, Maosong Sun, and Yang Liu. Self-knowledge guided retrieval augmentation for large language models. In Houda Bouamor, Juan Pino, and Kalika Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023, Singapore, December 6-10, 2023*, pages 10303–10315. Association for Computational Linguistics, 2023.
- [Wei *et al.*, 2025] Zhepei Wei, Wei-Lin Chen, and Yu Meng. Instructrag: Instructing retrieval-augmented generation via self-synthesized rationales. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025.
- [Wu *et al.*, 2025] Jinyang Wu, Shuai Zhang, Feihu Che, et al. Pandora's box or aladdin's lamp: A comprehensive analysis revealing the role of RAG noise in large language models. In Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, editors, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2025, Vienna, Austria, July 27 - August 1, 2025, pages 5019–5039. Association for Computational Linguistics, 2025.
- [Yan *et al.*, 2024] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. Corrective retrieval augmented generation. *CoRR*, abs/2401.15884, 2024.
- [Yang *et al.*, 2018] Zhilin Yang, Peng Qi, Saizheng Zhang, et al. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In Ellen Riloff, David Chiang, Julia Hockenmaier, and Jun'ichi Tsujii, editors, *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, October 31 - November 4, 2018*, pages 2369–2380. Association for Computational Linguistics, 2018.
- [Yang *et al.*, 2025] An Yang, Anfeng Li, Baosong Yang, et al. Qwen3 technical report. *CoRR*, abs/2505.09388, 2025.
- [Yu *et al.*, 2024] Wenhao Yu, Hongming Zhang, Xiaoman Pan, et al. Chain-of-note: Enhancing robustness in retrieval-augmented language models. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, EMNLP 2024, Miami, FL, USA, November 12-16, 2024*, pages 14672–14685. Association for Computational Linguistics, 2024.
- [Zhang *et al.*, 2025a] Qi Zhang, Shouqing Yang, Lirong Gao, et al. Lets: Learning to think-and-search via process-and-outcome reward hybridization. *CoRR*, abs/2505.17447, 2025.
- [Zhang *et al.*, 2025b] Yuxin Zhang, Yan Wang, Yongrui Chen, et al. Magic mushroom: A customizable benchmark for fine-grained analysis of retrieval noise erosion in RAG systems. *CoRR*, abs/2506.03901, 2025.
- [Zhou *et al.*, 2025] Huichi Zhou, Kin-Hei Lee, Zhonghao Zhan, Yue Chen, Zhenhao Li, Zhaoyang Wang, Hamed Haddadi, and Emine Yilmaz. Trustrag: Enhancing robustness and trustworthiness in RAG. *CoRR*, abs/2501.00879, 2025.

Appendix Contents

A	Details of Dataset Reconstruction	10
A.1	Reasoning Node Construction	10
A.2	Sub-Question Construction	10
A.3	Sub-Answer Reorganization	10
A.4	Golden Document Augmentation	11
A.5	Retrieval Policy Annotation	11
A.6	Intermediate Thought Synthesis	11
A.7	Manual Verification	11
B	Additional Details of ARMOR	11
B.1	Pseudo-Code of ARMOR	11
B.2	Proof of Unbiased Reward Estimation	12
C	Additional Experimental Setup	12
C.1	Baseline Configurations	12
C.2	Retrieval Noise Distribution	13
C.3	Experimental Prompts	13
D	Additional Experimental Results	13
D.1	Sensitivity Analysis	15
D.2	Case Study	15
E	Details of LLM-Based Answer Evaluation	15
E.1	Prompts for Correctness Evaluation	15
E.2	Human AI Alignment	15

A Details of Dataset Reconstruction

We reconstruct *2WikiMultiHopQA* and *MuSiQue* datasets to train the RAG generator and evaluate its performance on multi-hop noise-robust reasoning. As illustrated in Figure 4, ARMOR requires training instances organized as structured reasoning chains, where each node in reasoning chains contains an intermediate thought, a sub-question, a retrieval policy, a sub-answer and a set of golden documents supporting the sub-answer. However, the original versions of these datasets lack several node-level annotations. To bridge this gap, we reconstruct each QA instance into a fully structured reasoning chain through a seven-stage pipeline: (1) reasoning node construction, (2) sub-question construction, (3) sub-answer reorganization, (4) golden document augmentation, (5) retrieval policy annotation, and (6) intermediate thought synthesis (7) manual verification.

Several reconstruction stages rely on LLM-based generation. To mitigate potential stylistic bias arising from over-reliance on a single generator, we employ an ensemble of advanced LLMs, including GPT-5 [Singh *et al.*, 2025], Gemini-2.5-Pro [Comanici *et al.*, 2025], and DeepSeek-R1 [DeepSeek-AI, 2025], and we prompt multiple models and randomly sample from the pool of valid outputs. This strategy reduces the risk of introducing a discernible stylistic fingerprint into the synthetic data, thereby enhancing the ecological validity of the reconstructed datasets.

A.1 Reasoning Node Construction

This stage constructs a sequence of reasoning nodes. For *2WikiMultiHopQA*, a reasoning node is constructed from an evidence triple and its associated golden document. For *MuSiQue*, the dataset provides a sequence of reasoning steps through an entity and a relation phrase. We construct each reasoning node through pairing the entity and relation phrase with their annotated sub-answer and supporting document.

A.2 Sub-Question Construction

For each reasoning node, we employ LLMs to formulate the sub-question that precisely reflects its reasoning objective. In *2WikiMultiHopQA*, sub-questions are generated based on the evidence triples and the main-question. For the first node in reasoning chains, LLMs incorporate the main-question and evidence triple to generate the sub-question. For subsequent nodes, LLMs additionally incorporate previous sub-question-answer pairs to produce follow-up sub-questions that logically progress toward the final answer. The prompts used to generate the sub-question are shown in Figure 6 and Figure 5, respectively. In *MuSiQue*, LLMs refine each node’s entity and relation phrase into a complete sub-question, and the prompt is shown in Figure 7.

MuSiQue: Sub Question Reconstruction

Task Requirements:

You will be given one question in the format: *Entity >> Relation*. Your task is to convert each into a clear, natural-sounding English question. Use appropriate phrasing based on the relation (e.g., "distributed by", "owned by", etc.). Do not add any extra information, explanations, or formatting. Please output only the rephrased question.

Example Input:

The Game (1997 film) >> distributed by

Example Output:

Which company distributed The Game (1997 film)?

Input:

question: <question>

Figure 7: Prompt for reconstructing the sub question in MuSiQue.

A.3 Sub-Answer Reorganization

This stage employs LLMs to reorganize the sub-answers in *2WikiMultiHopQA* based on corresponding evidence triples. Specifically, sub-answers are normalized into concise phrases or sentences that directly address the corresponding sub-question. The prompt used for sub-answer reorganization is illustrated in Figure 8.

A.4 Golden Document Augmentation

In this stage, we leverage LLMs to paraphrase the original golden document of each node into five distinct variants. This process ensures that the augmented documents retain the semantic equivalence of the original while introducing linguistic diversity. These generated documents are utilized to systematically manipulate the golden document ratio and noise ratio in noise-robust experiments. The prompt used for document augmentation is detailed in Figure 10.

A.5 Retrieval Policy Annotation

We annotate each node by evaluating the RAG generator in retrieval-disabled mode. A node is labeled **retrieve-no** if the model generates a correct sub-answer based solely on its intrinsic knowledge and prior reasoning; otherwise, it is labeled **retrieve-yes**, indicating that external retrieval is required. This model-specific annotation provides explicit supervision for training the generator to make adaptive retrieval policies during the warm-up stage.

2WikiMultiHopQA: Sub Answer Reconstruction

Task Requirements:

You are an information conversion tool. Please convert the given triple [*subject*, *predicate*, *object*] into a grammatically correct and natural-sounding English sentence. Do not add any extra information, explanations, or formatting. Please output only the sentence.

Example Input:

[*novel Do Androids Dream of Electric Sheep?*, *basis*, *movie Blade Runner*]

Example Output:

The novel Do Androids Dream of Electric Sheep is the basis for the movie Blade Runner.

Input:

[<*subject*>, <*predicate*>, <*object*>]

Figure 8: Prompt for reconstructing the sub answer in 2WikiMultiHopQA.

A.6 Intermediate Thought Synthesis

This stage employ LLMs to synthesize an internal reasoning thought for each node, transforming the implicit logic into an explicit structured representation. Unlike simple concatenation of sub-questions and sub-answers, these states are generated by considering the global context, including the main question, the final answer, and the entire sequence of reasoning steps. Specifically, each synthesized thought elucidates the rationale behind its corresponding sub-question and justifies its necessity within the overall multi-hop chain, and the prompt is detailed in Figure 9. These synthesized thoughts

Algorithm 1 Adaptive Task Scheduler

Input: Task set $\mathcal{T}$, total training rounds T , training dataset $\mathcal{D}$

Hyperpara: Exploration coefficient γ , forgetting factor α

Output: Optimized RAG generator π_θ

```

1: Initialize learning feedback weight
    $w_1(\tau) \leftarrow \frac{1}{|\mathcal{T}|}, \forall \tau \in \mathcal{T}$ 
2: Initialize reward history
    $\mathcal{H}(\tau) \leftarrow \emptyset, \forall \tau \in \mathcal{T}$ 
3: for  $n = 1$  to  $T$  do
4:    $W_{sum} \leftarrow \sum_{\tau' \in \mathcal{T}} w_t(\tau')$ 
5:   for each task  $\tau \in \mathcal{T}$  do
6:      $p_n(\tau) \leftarrow (1 - \gamma) \frac{w_n(\tau)}{W_{sum}} + \frac{\gamma}{|\mathcal{T}|}$ 
7:   end for
8:   Sample a task  $\tau_s \sim p_n(\tau)$ 
9:   Select new batch of questions  $\mathcal{Q}$  from  $\mathcal{D}$ 
10:  Update  $\theta_G$  via inter group advantage estimation
    $\pi_\theta \leftarrow \text{Optimizer}(\pi_\theta, \tau_s, \mathcal{Q})$ 
11:  Observe task-level reward  $r_n(\tau_s)$ 
12:  if  $\mathcal{H}(\tau_s) \neq \emptyset$  then
13:     $r_{old}(\tau_s) \leftarrow \max\{r \in \mathcal{H}(\tau_s)\}$ 
14:     $ALP(\tau_s) \leftarrow |r_n(\tau_s) - r_{old}(\tau_s)|$ 
15:  else
16:     $ALP(\tau_s) \leftarrow 0$ 
17:  end if
18:   $\mathcal{H}(\tau_s) \leftarrow \mathcal{H}(\tau_s) \cup \{r_n(\tau_s)\}$ 
19:  for each task  $\tau \in \mathcal{T}$  do
20:    if  $\tau = \tau_s$  then
21:       $\hat{r}_n(\tau) \leftarrow ALP(\tau_s)/p_t(\tau_s)$ 
22:    else
23:       $\hat{r}_n(\tau) \leftarrow 0$ 
24:    end if
25:     $w_{n+1}(\tau) \leftarrow w_n(\tau) \cdot \exp\left(\frac{\gamma \hat{r}_n(\tau)}{|\mathcal{T}|}\right) + \frac{e \cdot \alpha}{|\mathcal{T}|} \cdot W_{sum}$ 
26:  end for
27: end for
28: return RAG generator  $\pi_\theta$ 

```

provide fine-grained supervision for the RAG generator to model internal reasoning processes during the warm-up stage.

A.7 Manual Verification

We conduct a manual verification stage to ensure the logical correctness of the reconstructed reasoning chains. Specifically, annotators inspect each chain to identify and remove cases with logical inconsistencies, invalid reasoning transitions, or misaligned sub-questions and sub-answers. This human-in-the-loop verification serves as a final quality control stage to guarantee the reliability of the reconstructed datasets.

B Additional Details of ARMOR

B.1 Pseudo-Code of ARMOR

In this appendix, we provide the pseudo-code for the key components of the ARMOR framework: **Adaptive Curriculum Scheduler** and **Inter-Group Advantage Estimation**. The Adaptive Curriculum Scheduler (Algorithm 1) dynamically select tasks for optimization during each training round. It begins by initializing the task weights and reward history. The scheduler computes task selection probabilities based on the current learning progress, balancing exploration and exploitation. The learning progress is assessed using ALP, and

task weights are updated after each round to reflect learning progress while retaining historical experience with a forgetting factor.

The Inter-Group Advantage Estimation (Algorithm 2) normalizes task-level and format-level rewards across the entire batch. Multiple rollouts are generated for each question, and rewards are assigned for each task span. These rewards are then normalized to compute task-level and format-level advantages. The policy model is updated using a clipped objective that incorporates the computed advantages, ensuring stable optimization despite task heterogeneity. Together, these two components enable ARMOR to provide an adaptive, stable, and efficient learning framework for noise-robust reasoning.

B.2 Proof of Unbiased Reward Estimation

In our adaptive curriculum meta-learning framework, The scheduler samples a task $\tau_s \sim p_n(\cdot)$ at the start of round n . Directly using the observed $ALP(\tau_s)$ to update task weights carries the risk of underestimating rarely selected tasks and overestimating frequently selected tasks. To ensure fair credit assignment across all tasks, we apply importance weighting to construct an unbiased estimation for $ALP(\tau)$. The estimated reward is defined as:

$$\hat{r}_n(\tau) = \frac{ALP(\tau)}{p_n(\tau)}. \quad (20)$$

Proof. $\hat{r}_n(\tau)$ is an unbiased estimation of $ALP(\tau)$.

$$\begin{aligned} \mathbb{E}[\hat{r}_n(\tau)] &= \mathbb{E}\left[\frac{ALP(\tau)}{p_n(\tau)}\right] \\ &= \frac{ALP(\tau)}{p_n(\tau)} \cdot p_n(\tau) \\ &= ALP(\tau). \end{aligned} \quad (21)$$

Hence, $\hat{r}_n(\tau)$ is an unbiased estimation for $ALP(\tau)$. $\square$

C Additional Experimental Setup

C.1 Baseline Configurations

To ensure a fair and consistent comparison, we employ unified LLM generators and evaluation metrics across all experiments, while adhering to the originally recommended hyperparameter settings for each baseline.

Baseline Settings. For DRAGIN, we follow the original configuration by setting the number of few-shot examples to six and the hallucination threshold to 1.0, thereby effectively evaluating the output confidence. Regarding IRCOT, we configure the maximum inference hops to four, with a retrieval limit of 15 documents per query. The generation of tokens is limited to a maximum of 248 tokens per hop. For ARMOR, we mask all `<doc>` tags and their associated documents during training to prevent information leakage. We employ LoRA with a rank of $r = 8$, a scaling factor of $\alpha = 16$, and disable dropout. For SEARCH-R1 and RAG-RL, we standardize the maximum sequence length to 1,500 tokens and perform six rollouts during the training process,

Algorithm 2 Iner Group Advantage Estimation

Input: Questions $\mathcal{Q}$, selected task τ_s , policy model π_θ , reward model π_{reward} ,
Hyperpara: Question number m , rollout number l , format weight ω , clipping coefficient ε , current round n
Output: Updated policy parameters θ

- 1: Initialize rollout set $\mathcal{O}_{\text{train}} \leftarrow \emptyset$
- 2: Sync $\pi_{\theta_{\text{old}}} \leftarrow \pi_\theta$
- 3: **for** $i = 1$ **to** m **do**
- 4: **for** $j = 1$ **to** l **do**
- 5: Sample rollout sequence $o_{i,j} \sim \pi_{\theta_{\text{old}}}(\cdot | q_i)$
- 6: $\mathcal{O}_{\text{train}} \leftarrow \mathcal{O}_{\text{train}} \cup \{o_{i,j}\}$
- 7: **end for**
- 8: **end for**
- 9: Initialize $R_{\text{task}} \leftarrow \emptyset$
- 10: **for each** $o_{i,j} \in \mathcal{O}_{\text{train}}$ **do**
- 11: Identify $K_{i,j}$ task-spans in $o_{i,j}$ for task τ_s
- 12: Compute rewards $\mathbf{r}_{i,j} = \{r_{i,j}^{(1)}, \dots, r_{i,j}^{(K_{i,j})}\}$ via π_{reward}
- 13: $R_{\text{task}} \leftarrow R_{\text{task}} \cup \{\mathbf{r}_{i,j}\}$
- 14: **end for**
- 15: Initialize $R_{\text{fmt}} \leftarrow \emptyset$
- 16: **for each** $o_{i,j} \in \mathcal{O}_{\text{train}}$ **do**
- 17: Compute format reward $r_{i,j}^{\text{fmt}}$
- 18: $R_{\text{fmt}} \leftarrow R_{\text{fmt}} \cup \{r_{i,j}^{\text{fmt}}\}$
- 19: **end for**
- 20: **for each** $o_{i,j} \in \mathcal{O}_{\text{train}}$ **do**
- 21: **for** $k = 1$ **to** $K_{i,j}$ **do**
- 22: $A_{i,j}^{\text{task}(k)} \leftarrow \frac{r_{i,j}^{(k)} - \text{mean}(R_{\text{task}})}{\text{std}(R_{\text{task}})}$
- 23: **end for**
- 24: $A_{i,j}^{\text{fmt}} \leftarrow \frac{r_{i,j}^{\text{fmt}} - \text{mean}(R_{\text{fmt}})}{\text{std}(R_{\text{fmt}})}$
- 25: **for** $t = 1$ **to** $|o_{i,j}|$ **do**
- 26: $A_{i,j}^{(t)} = \sum_{k=1}^{K_{i,j}} \mathbf{1}[t \in \text{tokens}(i, j, k)] \cdot A_{i,j}^{\text{task}(k)} + \omega A_{i,j}^{\text{fmt}}$
- 27: **end for**
- 28: **end for**
- 29: Initialize loss $\mathcal{L} \leftarrow 0$
- 30: **for** $i = 1$ **to** m **do**
- 31: $L_i \leftarrow 0, N_i \leftarrow 0$
- 32: **for** $j = 1$ **to** n **do**
- 33: **for** $t = 1$ **to** $|o_{i,j}|$ **do**
- 34: Compute likelihood ratio:

$$\rho_{i,j}^{(t)} \leftarrow \frac{\pi_\theta(o_{i,j}^{(t)} | q_i, o_{i,j}^{(<t)})}{\pi_{\theta_{\text{old}}}(o_{i,j}^{(t)} | q_i, o_{i,j}^{(<t)})}$$
- 35: Clipped objective:

$$c_{i,j}^{(t)} \leftarrow \min\left(\rho_{i,j}^{(t)} A_{i,j}^{(t)}, \text{clip}(\rho_{i,j}^{(t)}, 1 - \varepsilon, 1 + \varepsilon) \cdot A_{i,j}^{(t)}\right)$$
- 36: $L_i \leftarrow L_i + c_{i,j}^{(t)}$
- 37: $N_i \leftarrow N_i + 1$
- 38: **end for**
- 39: **end for**
- 40: $\mathcal{L} \leftarrow \mathcal{L} + \frac{L_i}{N_i}$
- 41: **end for**
- 42: Update θ by maximizing $\mathcal{L}$
- 43: Rollout on validation set using π_θ
- 44: Compute $r_n(\tau_s) \leftarrow \text{mean}(R_{\text{task}}^{\text{val}})$
- 45: **return** policy mode $\pi_\theta, r_n(\tau_s)$

maintaining the same LoRA configuration as ARMOR. During the rollout process, we adopt a sampling temperature of 0.85, complemented by top- $p = 0.95$ and top- $k = 50$. The retrieval module is configured to retain the top three documents for each query. Specifically for RAG-RL, we implement the min-max curriculum strategy, which is empirically identified as the optimal setting in our preliminary study. In addition, it is crucial to note that SEARCH-R1 and RAG-RL are initialized directly from the original instruct-tuned model, rather than the model processed through our warm-up stage. This design choice is motivated by the fact that our warm-up stage is specifically engineered to instill particular reasoning formats. Since SEARCH-R1 and RAG-RL adhere to their own distinct structural paradigms, transferring the format biases from our warm-up model would create conflicting learning objectives, resulting in training collapse.

Inference and Environment. During the inference phase, all baselines and ARMOR utilize `e5-base-v2` as the unified retriever, retrieving up to three documents per iteration from the consistent corpus. Sampling hyperparameters (*i.e.*, temperature, top- k , and top- p) follow the default settings of the original instruct-tuned models. All experiments are conducted on four NVIDIA A100 GPUs. To ensure statistical reliability, reported results are averaged over three independent runs (using random seed 42 for initialization).

C.2 Retrieval Noise Distribution

Standard RAG systems inevitably retrieve irrelevant or distracting documents, which introduces *natural noise* into the generation process. To quantify this phenomenon, we analyze the composition of retrieved documents across *2Wiki*, *MuSiQue*, and *HotpotQA* using `e5-base-v2` as the retriever with retrieve $k = 3$. Specifically, for both the original multi-hop questions and their decomposed sub-questions, we categorize each retrieved document into three types: **Golden** (directly supports the answer), **Inconsequential** (semantically related but not essential), and **Irrelevant** (unrelated to the question).

Datasets	Golden	Inconsequential	Irrelevant
Multi-Hop Question			
<i>2Wiki</i>	21.87	30.03	48.10
<i>MuSiQue</i>	12.92	35.65	51.43
<i>HotpotQA</i>	17.14	33.89	48.97
Decomposed Sub-Question			
<i>2Wiki</i>	25.06	29.17	45.77
<i>MuSiQue</i>	16.67	34.55	48.78
<i>HotpotQA</i>	13.33	33.67	53.00

Table 4: Distribution of retrieved document types for multi-hop questions and their decomposed sub-questions across three datasets.

The results in Table 4 confirm that **retrieval noise is inherent in standard RAG pipelines**: a significant fraction of retrieved passages are either inconsequential or irrelevant. Multi-hop questions generally exhibit higher noise ratios than their decomposed sub-questions, underscoring the

greater challenge posed by compositional questions to RAG retrievers. Furthermore, the retrieval noise ratios exhibit a progressive increase across three datasets: *2WikiMultiHopQA* shows the lowest noise ratio while *MuSiQue* and *HotpotQA* exhibit higher noise ratios, correlating with growing reasoning complexity or entity diversity. These findings highlight the necessity of noise-robust reasoning for multi-hop QA.

C.3 Experimental Prompts

This section provides an overview of the prompts employed during the inference stage of ARMOR. Figure 12 illustrates the inference prompt used in ARMOR. When the model generates the token sequence `<search>True</search>`, the retriever is activated to fetch relevant documents based on the preceding `<sub-question>`. Each retrieved document is then inserted sequentially into the context; after each insertion, the RAG generator produces a corresponding `<doc-type>` annotation. In addition, Figure 11 shows the prompt template used to generate answers without retrieval. This variant is not part of the main inference pipeline but is instead utilized during the reinforcement learning phase to train the Knowledge Boundary Perception task, where the model learns to recognize when its internal knowledge suffices to answer a question.

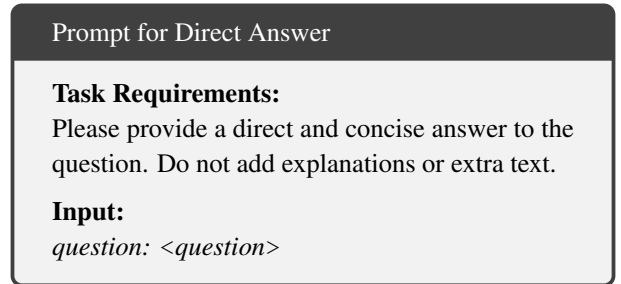


Figure 11: Prompt for direct answer.

D Additional Experimental Results

As shown in Table 5, small-scale language models typically struggle with complex multi-hop reasoning and are highly susceptible to retrieval noise due to their limited instruction-following and reasoning capabilities. For instance, on Qwen-3-0.6B, advanced RL-based methods like SEARCH-R1 collapse, achieving an average correctness of only 19.66%, which is even lower than the NO-RAG baseline (20.48%). This phenomenon indicates that standard RL optimization is unstable for weaker generic models. In stark contrast, ARMOR demonstrates exceptional efficacy on lightweight models. On Qwen-3-0.6B, ARMOR achieves an average correctness of 57.22%, outperforming IRCOT. Similarly, ARMOR surpasses IRCOT by 12.89% in average correctness on Llama-3.2-1B. These results confirm that ARMOR’s adaptive curriculum learning acts as an effective *teacher*, enabling even smaller *student* generators to master robust reasoning capabilities that are otherwise inaccessible through standard training.

Method	2WikiMultiHopQA			MusiQue			HotpotQA			Avg.		
	Cor.	F1	EM	Cor.	F1	EM	Cor.	F1	EM	Cor.	F1	EM
Qwen-3-0.6B												
No-RAG	37.43	35.55	32.99	3.73	2.58	0.00	20.27	16.26	10.47	20.48	18.13	14.49
VANILLA-RAG	34.47	28.16	21.15	14.67	11.63	2.67	34.87	28.24	21.27	28.00	22.68	15.03
CoT	34.17	31.29	28.55	2.67	5.99	0.00	19.67	18.16	14.33	18.84	18.48	14.29
CoN	56.15	19.33	9.91	19.73	15.98	8.27	46.13	32.50	26.53	40.67	22.60	14.90
IRCoT	36.69	31.06	23.52	13.33	12.44	3.73	42.47	35.57	26.87	30.83	26.36	18.04
DRAGIN	32.40	33.10	30.47	1.33	3.76	0.00	19.60	18.97	16.53	17.78	18.61	15.67
SKR	36.83	33.53	29.29	12.00	4.72	0.00	24.87	23.03	16.60	24.57	20.43	15.30
RAG-RL	37.58	31.24	25.49	18.73	15.82	3.47	36.97	32.22	25.67	31.09	26.43	18.21
SEARCH-R1	24.18	17.35	11.55	8.67	5.89	1.67	26.13	19.35	12.47	19.66	14.20	8.56
ARMOR (warm-up stage)	62.57	62.49	59.32	28.00	28.97	21.33	32.27	31.03	33.60	40.95	40.83	38.08
ARMOR	72.34	73.31	70.56	49.33	47.40	34.67	50.00	49.43	41.13	57.22	56.71	48.79
Qwen-3-4B												
No-RAG	32.84	28.41	23.67	10.67	11.08	2.67	30.07	25.64	20.13	24.53	21.71	15.49
VANILLA-RAG	66.12	63.75	57.99	24.00	21.13	16.00	58.67	56.64	50.13	49.60	47.17	41.37
CoT	44.97	28.53	23.52	12.00	9.16	2.67	33.27	30.88	25.27	30.08	22.86	17.15
CoN	65.83	24.19	17.16	36.00	25.64	16.00	59.13	45.12	38.57	53.65	31.65	23.91
IRCoT	67.31	62.72	55.03	44.00	34.35	24.00	63.93	57.60	48.33	58.41	51.56	42.45
DRAGIN	45.19	46.50	43.64	8.00	9.23	4.00	32.87	32.84	29.73	28.69	29.52	25.79
SKR	36.24	25.44	18.05	16.00	8.56	1.33	30.00	22.86	16.60	27.41	18.95	11.99
RAG-RL	69.26	70.15	68.79	25.67	22.26	18.00	61.27	59.64	59.40	52.07	50.68	48.73
SEARCH-R1	59.56	56.90	51.74	21.67	18.22	13.13	50.67	48.75	47.33	43.97	41.29	37.40
ARMOR (warm-up stage)	70.12	70.85	69.38	24.00	24.29	14.67	51.07	50.38	44.20	48.40	48.51	42.75
ARMOR	77.96	77.44	75.00	46.67	46.88	36.00	63.13	63.13	56.40	62.59	62.48	55.80
Llama-3.2-1B												
No-RAG	32.69	24.15	14.94	1.33	2.68	0.00	25.20	17.84	12.40	19.74	14.89	9.11
VANILLA-RAG	47.63	44.49	37.28	17.33	19.47	10.67	45.47	41.35	36.53	36.81	35.10	28.16
CoT	26.33	3.85	0.15	0.00	1.63	0.00	16.27	6.75	1.00	14.20	4.08	0.38
CoN	34.91	7.20	0.89	18.67	3.58	0.00	35.07	5.94	1.20	29.55	5.57	0.70
IRCoT	49.85	44.40	35.80	21.33	23.79	14.67	45.00	44.27	37.40	38.73	37.49	29.29
DRAGIN	36.24	36.64	34.62	6.67	8.37	4.00	21.60	20.19	17.00	21.50	21.73	18.54
SKR	31.95	22.63	14.64	0.00	2.39	0.00	19.73	19.87	14.66	17.23	14.96	9.77
RAG-RL	51.35	48.56	41.28	18.67	19.60	12.33	49.53	41.35	41.87	39.85	36.50	31.83
SEARCH-R1	45.52	42.62	35.39	15.33	17.52	8.67	42.73	41.35	33.40	34.53	33.83	25.82
ARMOR (warm-up stage)	67.01	66.80	64.35	30.67	35.19	24.00	42.53	41.39	34.20	46.74	47.79	40.85
ARMOR	74.11	74.52	72.78	30.67	33.08	26.67	50.07	49.48	42.00	51.62	52.36	47.15

Table 5: Performance comparing our method with baselines on three multi-hop QA datasets using Qwen-3-0.6B, Qwen-3-4B, Llama-3.2-1B as backbones. We report Correctness (Cor.), F1 score, and Exact Match (EM) in percentage (%). The best results are highlighted in **bold**.

D.1 Sensitivity Analysis

We conduct sensitivity analysis using Qwen-3-1.7B to evaluate ARMOR’s stability under varying hyperparameter settings. As illustrated in Figure 14, the generator’s performance exhibits a trend of initially increasing and then decreasing as K (the number of retrieved documents) increases. Performance peaks at $K = 3$, suggesting that a moderate amount of context provides sufficient information for reasoning. However, further increasing does not yield continuous improvements. This phenomenon indicates that retrieval is not “the more, the better.” Introducing an excessive number of documents significantly increases the risk of incorporating noise, which overwhelms the generator’s reasoning capabilities and dilutes the signal from golden documents.

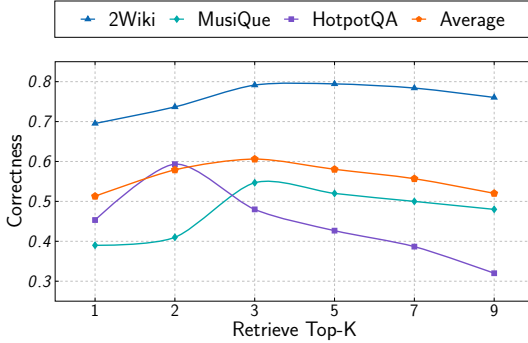


Figure 14: Impact of retrieval size (K) on generation correctness using Qwen-3-1.7B.

We analyze the impact of rollout size (N) on training stability and performance. Figure 13 compares ARMOR against the variant without Inter-Group Advantage Estimation (*w/o IGAE*). ARMOR exhibits remarkable sample efficiency, maintaining high correctness, F1, and EM scores even with a minimal rollout number ($N = 1$). In contrast, the *w/o IGAE* variant is highly sensitive to N , performing poorly when N is small. With a small N and the constrained output space of sub-tasks, the generated outputs within a single group often become identical. By leveraging Inter-Group Advantage Estimation, ARMOR expands the comparison scope to the entire batch, ensuring effective advantage estimation regardless of group size.

D.2 Case Study

Table 6 presents a concrete example of multi-hop question answering with different reasoning method. We illustrate how each method decomposes the question, interacts with retrieved evidence, and concludes a final answer. The case highlights distinct failure modes in SEARCH-R1 and IRCOT. In contrast, ARMOR demonstrates more structured reasoning and effective noise filtering from retrieved contexts.

E Details of LLM-Based Answer Evaluation

E.1 Prompts for Correctness Evaluation

In addition to traditional lexical metrics such as Exact Match and F1, we employ an LLM-based scoring approach using

GPT-5 to evaluate the answer correctness in RAG systems. As illustrated in Figure 15, our evaluation prompt instructs the model to assess the factual accuracy and completeness of generated answers by comparing them against reference answers. This approach overcomes the limitations of surface-level matching inherent in traditional QA metrics. Moreover, we also leverage this LLM-based evaluation to compute rewards for robust reasoning task during RL training, providing more informative feedback to guide the model compared with traditional lexical metrics.

E.2 Human AI Alignment

To validate the alignment between LLM-based automatic evaluation and human judgment, we conducted an alignment analysis on the performance of Qwen-3-1.7B at different noise ratios using VANILLARAG as the framework. We randomly sampled 100 instances from *2WikiMultiHopQA* and *MuSiQue* at different noise ratios. For each instance, we computed the average correctness assigned independently by human evaluators and GPT-5. As shown in Figure 16, the scores exhibit high similarity, indicating strong alignment between LLM-based and manual judgements. These findings support the effectiveness of the LLM-based evaluation framework as a reliable surrogate for human judgment in large-scale experimental settings.

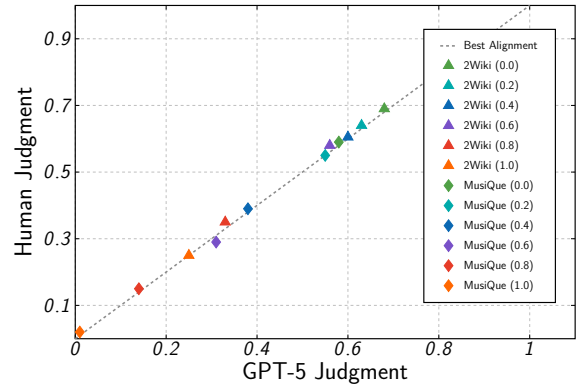


Figure 16: Comparison of correctness based on (1) Human and (2) GPT-5 Judgement. Legend entries such as “2Wiki (0.6)” denote results of using *2WikiMultiHopQA* with 60% noise ratio.

Table 6: Comparative case study of ARMOR, SEARCH-R1, and IRCOT on multi-hop QA with noisy retrieval.

Question:	When was Ole Rynning’s father born?
Ground Truth:	May 30, 1778
ARMOR:	<think> To find out when Ole Rynning’s father was born, I first need to identify who Ole Rynning’s father is. </think> <sub-question> Who is Ole Rynning’s father? </sub-question> <search> True </search> <doc> ... He was the son of the priest Jens Rynning (1778–1857) ... </doc> <doc-type> golden </doc-type> <doc> ... </doc> <doc-type> irrelevant </doc-type> ... <sub-answer> Ole Rynning’s father is Jens Rynning. </sub-answer> <think> Now that I know Ole Rynning’s father is Jens Rynning, I need to find his date of birth. </think> <sub-question> When was Jens Rynning born? </sub-question> <search> True </search> <doc> Jens Rynning (May 30, 1778 – June 11, 1857) was a ... </doc> <doc-type> golden </doc-type> <doc> Ole Rynning (April 4, 1809 – September 1838) was a ... </doc> <doc-type> inconsequential </doc-type> ... <sub-answer> Jens Rynning was born on May 30, 1778. </sub-answer> <main-answer> May 30, 1778. </main-answer>
SEARCH-R1:	<think> Okay, I need to find out when Ole Rynning’s father was born. First, I should check if I have any existing information about Ole Rynning. I remember that Ole Rynning is a Danish footballer... </think> <search> Ole Rynning birth date </search> <information> ... He was the son of the priest Jens Rynning (1778–1857) ... </information> <search> Ole Rynning father birth date </search> <information> ... He was born in Stavanger as a son of Ole Bergesen ... </information> <answer> Jens Rynning </answer>
IRCOT:	<i>After interleaving chain-of-thought reasoning with iterative retrieval, the QA reader generates the final answer:</i> <doc> Jens Rynning (May 30, 1778 – June 11, 1857) was a ... </doc> <doc> Ole Rynning (April 4, 1809 – September 1838) was a ... </doc> <doc> ... He was the son of the priest Jens Rynning (1778–1857) ... </doc> <doc> ... </doc> ... <main-answer> April 4, 1809. </main-answer>

Structured Reasoning Chain

Main Question: When did the director of film The Argyle Secrets die?

Main Answer: April 16, 1995

Reasoning Node 1:

- **Intermediate Thought:** To find out when the director of The Argyle Secrets died, I first need to identify who directed the film.
- **Sub-Question:** Who directed the film The Argyle Secrets?
- **Sub-Answer:** The Argyle Secrets was directed by Cy Endfield.
- **Retrieval Policy:** retrieve-then-answer
- **Golden Documents:**
 - (a) The Argyle Secrets is a 1948 American mystery film directed by Cy Endfield and starring William Gargan and Marjorie Lord.
 - (b) In 1948, The Argyle Secrets, an American mystery film, was brought to life by director Cy Endfield, with notable performances from William Gargan and Marjorie Lord in leading roles.
 - (c) The Argyle Secrets, released in 1948, is an American mystery film directed by Cy Endfield, featuring performances by William Gargan and Marjorie Lord in the lead roles.
 - (d) ...

Reasoning Node 2:

- **Intermediate Thought:** Now that I know Cy Endfield directed The Argyle Secrets, I can look up his date of death to answer the main question.
- **Sub-Question:** When did Cy Endfield die?
- **Sub-Answer:** The Argyle Secrets was directed by Cy Endfield.
- **Retrieval Policy:** direct-answer
- **Golden Documents:**
 - (a) TCyril Raker Endfield (November 10, 1914 – April 16, 1995) was an American screenwriter, film director, theatre director, author, magician and inventor.
 - (b) Cyril Raker Endfield, born on November 10, 1914, and passing away on April 16, 1995, was a multifaceted American creative talent who excelled in various fields including screenwriting, filmmaking, theatre direction, authoring books, performing magic, and inventing.
 - (c) ...

Figure 4: An example of the structured reasoning chain..

2WikiMultiHopQA: First-Node Sub Question Generation

Task Requirements:

You are a reasoning assistant tasked with reconstructing the reasoning chain for a multi-hop question. Your job is to generate **one natural language sub-question** based on the given inputs:

- **Main Question**
- **Main Answer**
- **Evidence** (formatted as a triple: *[subject, predicate, object]*)

The sub-question must:

- Be answerable using **only** the given evidence triple.
- Serve as a logical intermediate step toward the main question.
- Explicitly mention the subject when appropriate, and use clear, concise language.

Output **strictly** in the following **JSON** format. Do NOT add explanations or extra text.

Example Input:

main question: Who wrote the novel that inspired the film Blade Runner?

main answer: Philip K. Dick

evidence: [novel Do Androids Dream of Electric Sheep?, basis, movie Blade Runner]

Example Output:

```
{ "sub-question": "What novel is the basis for the movie Blade Runner?" }
```

Input:

main question: <main question>

main answer: <main answer>

evidence: <evidence>

Figure 5: Prompt for generating the first-hop sub question in 2WikiMultiHopQA.

2WikiMultiHopQA: Multi-Hop Sub Question (2nd-Node Onward) Generation

Task Requirements:

You are a reasoning assistant tasked with reconstructing the reasoning chain for a multi-hop question. Your job is to generate **one natural language sub-question** based on the given inputs:

- **Main Question**
- **Main Answer**
- **Evidence**

(formatted as a triple: *[subject, predicate, object]*)

- **Previous Sub Question-Answer Pair**

(formatted as: *[[sub-question-1, sub-answer-1], [sub-question-2, sub-answer-2], ...]*)

The sub-question must:

- Be focused and answerable using **only** the given supporting fact.
- Together with prior sub Question-Answer steps, Represent a **logical intermediate step** toward answering the main question.
- Use **clear and concise language**, explicitly mentioning the ****subject**** when appropriate.

Output **strictly** in the following **JSON** format. Do NOT add explanations or extra text.

Example Input:

main question: Who wrote the novel that inspired the film Blade Runner?

main answer: Philip K. Dick

evidence: ["Do Androids Dream of Electric Sheep?", "author", "Philip K. Dick"]

previous sub question-answer pair:

[["What novel is the basis for the movie Blade Runner?", "Do Androids Dream of Electric Sheep?"]]

Example Output:

{ "sub-question": "What is the name of the author of Do Androids Dream of Electric Sheep?" }

Input:

main question: <main question>

main answer: <main answer>

evidence: <evidence>

previous sub question-answer pair: <previous sub question-answer pair>

Figure 6: Prompt for generating multi-hop sub-questions (second-hop onward) in 2WikiMultiHopQA.

Thinking Process Reconstruction

Objective:

You are a reasoning assistant designed to reconstruct the implicit thinking process in a multi-hop reasoning chain. Given a main question, its final answer, and a sequence of reasoning steps, each includes a sub-question and sub-answer pair, your task is to generate a concise internal thought before each reasoning step. Each internal thought should:

- Reflect the reasoning or purpose behind the corresponding sub-question.
- Explain why this step is necessary in the context of the overall reasoning process.
- Logically connect to the previous steps when applicable.
- Be succinct and focused on the rationale for that specific step.

Task Requirements:

- Input is provided in **JSON** format with fields: “*main-question*”, “*main-answer*”, and “*reasoning-steps*” (a list of “*sub-question*” and “*sub-answer*”).
- Output must be a **JSON** format containing only the generated “*thought*” strings, in the same order as the reasoning steps.
- Each “*thought*” must:
 - (a) Explain the rationale behind the corresponding sub-question.
 - (b) Be concise and logically connected to the overall chain.
 - (c) Do **not** return the original input or duplicate any fields.

Example Input:

```
{
  "main-question": "Who wrote the novel that inspired the film Blade Runner?",
  "reasoning-steps": [
    {
      "sub-question": "What novel is the basis for the movie Blade Runner?",
      "sub-answer": "The novel Do Androids Dream of Electric Sheep? is the basis for the movie Blade Runner."
    },
    {
      "sub-question": "What is the name of the author of Do Androids Dream of Electric Sheep?",
      "sub-answer": "The novel Do Androids Dream of Electric Sheep? was written by Philip K. Dick."
    }
  ],
  "main-answer": "Philip K. Dick"
}
```

Example Output:

```
{
  "step-1": "To identify the author of the novel that inspired Blade Runner, I first need to determine which novel served as the source material for the film.",
  "step-2": "Now that I know the novel is Do Androids Dream of Electric Sheep?, I need to find its author to answer the main question."
}
```

Input:

```
{
  "main-question": <main question>,
  "reasoning-steps": <reasoning steps>,
  "main answer": <main answer>
}
```

Figure 9: Prompt for reconstructing the thinking process in 2WikiMultiHopQA and MusiQue.

Prompt for Golden Document Augmentation

Task Requirements:

Please generate *<#document number>* documents based on the input document below. Each generated document should have the same meaning as the input document but be rephrased differently. Return the results in JSON format where the keys are integers from 1 to *<#document number>*, and the values are the rephrased documents.

Document:

<document>

Example Input:

The Radio station currently plays a mix of Hindi and Regional music.

Example Output:

```
{  
  1: "The Radio station is broadcasting a combination of Hindi and Regional music.",  
  2: "A mix of Hindi and Regional music is being played on the Radio station.",  
  3: "The Radio station features both Hindi and Regional music in its current playlist."  
}
```

Figure 10: Prompt for golden document augmentation.

Inference Prompt of ARMOR

Task Requirements:

You are a reasoning assistant. Answer `<main-question>` by performing explicit, step-by-step reasoning. Follow this structure **exactly**, and do **not** include any text outside these tags:

1. `<think> ... </think>` — Describe your internal reasoning process for the current step.
2. `<sub-question> ... </sub-question>` — State the next sub-question needed to continue reasoning.
3. `<search> True </search>` or `<search> False </search>` — Indicate whether external retrieval is required for this sub-question.
4. `<doc> ... </doc>` — Include the full content of any retrieved documents. Skip if none.
5. `<doc-type>golden</doc-type>` or `<doc-type>inconsequential</doc-type>` `<doc-type>irrelevant</doc-type>` — Classify each retrieved document.
6. `<sub-answer> ... </sub-answer>` — Provide the answer to the current sub-question.

Repeat steps 1–6 as needed for multiple reasoning steps. Finally, use `<main-answer> ... </main-answer>` to provide the final answer to the main question.

Example:

```
<main-question>Who is younger, Marvin E. Miller Sr. or John Hawkins (Author)?</main-question>
<think>Find the birth years of both people.</think>
<sub-question>When was Marvin E. Miller Sr. born?</sub-question>
<search>True</search>
<doc>Marvin E. Miller Sr. was born in 1917.</doc> <doc-type>useful</doc-type>
<sub-answer>1917</sub-answer> <think>Find John Hawkins' birth year.</think>
<sub-question>When was John Hawkins (Author) born?</sub-question>
<search>False</search>
<sub-answer>1960</sub-answer> <think>Compare birth years; the later year is younger.</think>
<main-answer>John Hawkins (Author) is younger.</main-answer>
```

Input:

```
<main-question (tag)> <main question> </main-question (tag)>
```

Figure 12: Inference prompt of ARMOR.

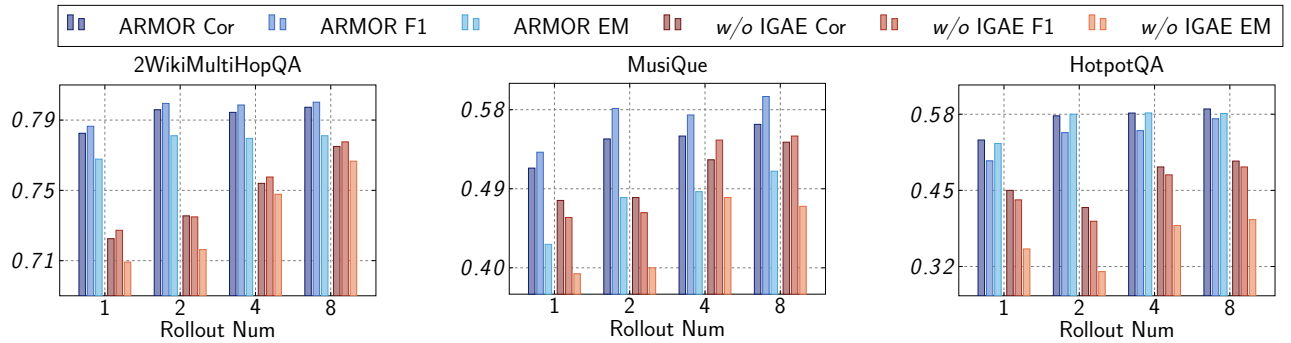


Figure 13: Sensitivity analysis of rollout number (N) on training stability using Qwen-3-1.7B.

Prompt for the Evaluation of Correctness

You are an evaluator tasked with scoring the Candidate Answer.

Instructions:

1. Compare the Candidate Answer to the Correct Answer in the context of the Question.
2. Assign a score from 0 to 5 based on accuracy and completeness:

Score 0: Completely incorrect or irrelevant.

Score 1: Partially correct but contains significant errors.

Score 3: Moderately correct but lacks some details or precision.

Score 5: Fully correct and matches the Correct Answer.

3. Output ONLY the score as a single number (e.g. “3”). Do not include any explanations.

Question: <question>

Correct Answer: <correct answer>

Candidate Answer: <llm answer>

Figure 15: Prompt for the evaluation of correctness.